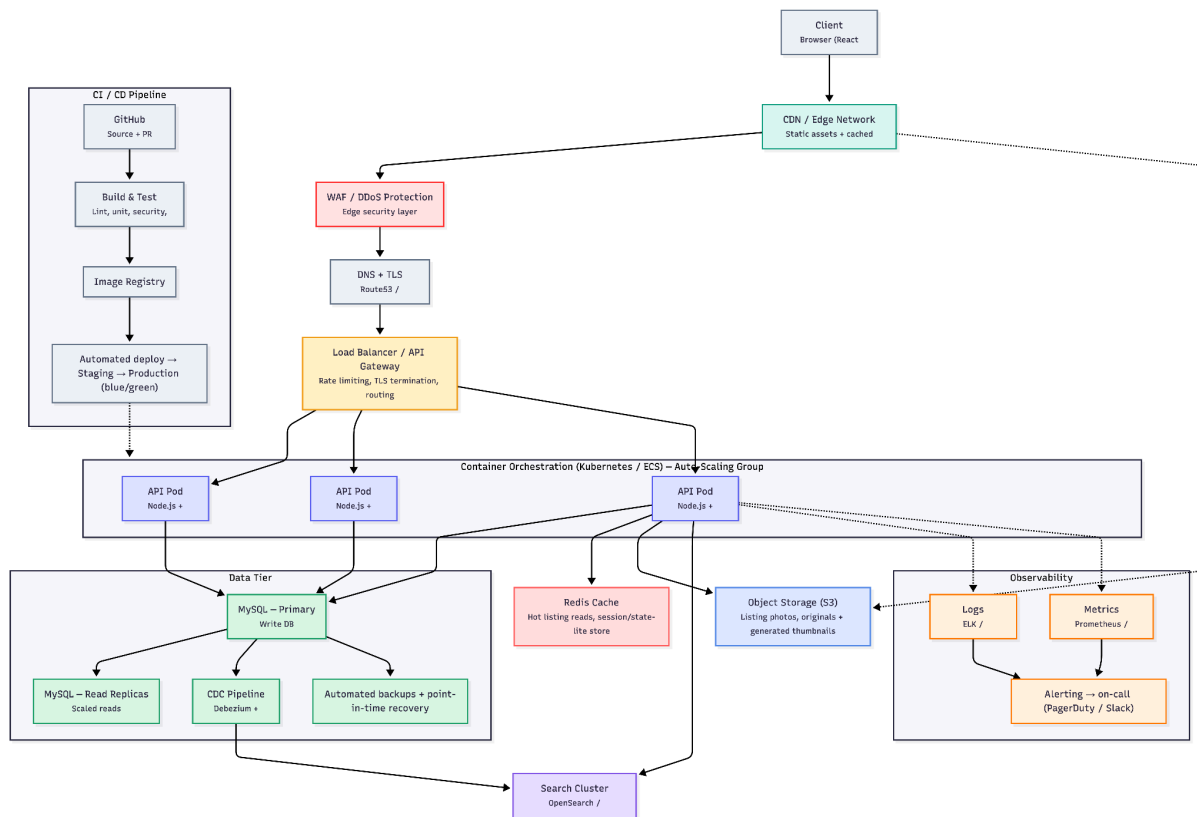


Architecture Report — Vacation-Rental Marketplace (Production-Scale)



This report explains each component in the architecture diagram and the reasoning behind including it. It's organized in the same six layers as the diagram: Client, Edge & Security, Application, Async & Caching, Persistence, and Platform/DevOps.

1. Client Layer

React SPA (Web) The listing page, Photo Tour, and Lightbox are all built as a single-page React app. A SPA is used (rather than server-rendered pages) because the interactions the assignment grades — modal overlays, keyboard navigation, hover/transition states — are inherently client-side and benefit from React's component/state model, especially for the lightbox's prev/next/focus-trap logic.

Future: Mobile App Not part of the current build (the assignment is desktop-only), but included to show the architecture doesn't paint itself into a corner — the same backend API tier can serve a future React Native client without any backend redesign, since the API doesn't assume a browser.

2. Edge & Security

CDN / Edge Cache Listing photos and static JS/CSS bundles are the heaviest, most repeatedly-requested assets on a page like this. Serving them from an edge CDN (CloudFront/Vercel Edge) instead of the origin server cuts load times globally and removes that traffic from the application servers entirely.

WAF + DDoS Shield A public-facing marketplace is a target for scraping, credential-stuffing on login, and volumetric attacks. A WAF filters malicious request patterns before they reach the app; a DDoS shield absorbs traffic spikes that aren't legitimate users, so the app tier doesn't have to scale to survive an attack.

Load Balancer Distributes incoming requests across multiple API instances and terminates TLS in one place, rather than every app instance handling certificates individually. This is what makes horizontal scaling of the app tier actually usable.

API Gateway Centralizes rate limiting, request routing to the right service, and a first pass of auth-token verification. Without this, every individual service would need to reimplement rate limiting and routing logic — the gateway keeps that logic in one place.

3. Application Tier (stateless, containerized, auto-scaled)

Core API Service Serves listing details, photos, and amenities — the read-heavy endpoints that power the Listing Page, Photo Tour, and Lightbox. Kept as its own service because it's the highest-traffic, lowest-write path in the system, and can be scaled independently of lower-traffic services like auth.

Auth Service Handles register/login/logout/session-check. Split out from the Core API because auth has different security requirements (rate limiting, password hashing cost, stricter validation) and a different traffic pattern (spiky around login, not constant like listing reads) — isolating it limits the blast radius if it needs patching or gets targeted by brute-force attempts.

Wishlist Service Handles the save/heart toggle. Small and simple by design — separated mainly so it can be scaled or modified independently without redeploying the whole Core API, and because it's the one place per-user write traffic happens on an otherwise read-dominated page.

Search API Handles query/filter/geo-search. Deliberately separate from Core API because search queries hit a completely different data store (see OpenSearch below) and have very different performance characteristics (full-text/geo relevance ranking vs. simple primary-key lookups).

Media Upload Service Issues presigned upload URLs and validates image metadata before a host's photos land in storage. Kept separate so that large file-handling logic and validation never blocks or slows down the fast-read Core API path.

Why stateless + containerized + auto-scaled: Every service stores no session state locally (auth is JWT-based), so any instance can handle any request. That's what allows the orchestrator (ECS/Kubernetes) to add or remove instances under load without any user-visible disruption — a service can crash or scale down and no in-flight session is lost.

4. Async Processing & Caching

Message Queue (SQS/Kafka) Some work triggered by a user action shouldn't block the response — e.g., after a host uploads a photo, resizing it into multiple sizes doesn't need to finish before the API responds "upload successful." The queue decouples the fast, synchronous user-facing request from slower background work.

Background Workers Consume the queue to do the actual heavy lifting: image resizing, updating the search index, sending notification emails. Scaled independently of the API tier since their load (a burst of new listings) doesn't correlate with browsing traffic.

Redis Cluster Caches the hottest reads — popular listing details, session/rate-limit counters — in memory. A listing page like this gets read far more often than it's written, so caching cuts MySQL load dramatically and drops response times from tens of milliseconds to sub-millisecond for cache hits.

CDC Stream (Debezium) Change Data Capture watches MySQL's write log and streams changes into OpenSearch automatically. This is used instead of having the app write to both MySQL and OpenSearch directly, because dual-writes are a common source of data-consistency bugs (one write can succeed while the other fails) — CDC guarantees the search index eventually reflects the source of truth without the app needing to manage that itself.

5. Persistence — Data, Search, Media

MySQL Primary The single source of truth for listings, users, and wishlists. All writes go here so there's never ambiguity about which copy of the data is authoritative.

MySQL Read Replicas Asynchronous copies of the primary, used for read traffic (which is the vast majority of traffic on a listing page). This lets read load scale horizontally by adding more replicas, without ever risking a write conflict, since writes only ever go to the primary.

Connection Pooler (RDS Proxy / ProxySQL) With many stateless, auto-scaled app instances, each opening its own raw DB connections would quickly exhaust MySQL's

connection limit. A pooler multiplexes many app-level requests over a smaller, managed set of real DB connections.

OpenSearch / Elasticsearch MySQL is good at exact lookups (`WHERE id = ?`) but poor at full-text relevance ranking, fuzzy matching, and geo-radius queries — all things a real "search listings near me" feature needs. A dedicated search engine handles that class of query efficiently, kept in sync from MySQL via CDC rather than being the source of truth itself.

S3 Object Storage Photos are large binary files — storing them as blobs in MySQL would bloat the database, slow backups, and waste an expensive resource (relational storage) on something object storage is purpose-built and far cheaper for. MySQL only stores the photo's URL, not its bytes.

Image CDN Sits in front of S3 to serve resized/optimized variants (thumbnail, hero, full-size) close to the user, rather than the origin bucket handling every image request directly.

6. Platform, CI/CD & Observability

GitHub Actions CI/CD Automates build, test, and deploy on every merge, so releases are consistent and repeatable rather than manual and error-prone.

ECS / Kubernetes Orchestrates the containerized services — handles auto-scaling, restarts failed instances, and rolls out new versions with zero downtime (blue-green/rolling deploys).

Staging + Prod Environments Changes are validated against a staging environment (with realistic but non-production data) before reaching real users, catching issues without risking the live marketplace.

Logging / Metrics / Tracing Centralized logging and metrics (and distributed tracing across services) are what make a multi-service architecture debuggable — without this, tracking a slow request across five services would be guesswork.

Secrets Manager + VPC DB credentials, JWT signing keys, and API keys are never stored in code or plain environment files in this model — they're pulled at runtime from a secrets manager. The VPC keeps the database and cache in private subnets, unreachable directly from the public internet; only the app tier (inside the same VPC) can reach them.
